

Fraud Detection in Financial Transactions

Author: Onkar Swaroop

Batch: Data Analytics [December 2024]

Email: onkarswaroop98@gmail.com

Introduction

Financial fraud is a growing concern, with digital transactions increasing rapidly across banking, e-commerce, and payment platforms. Fraudulent activities not only lead to **financial losses** but also erode customer trust, making fraud detection a critical challenge for businesses. Traditional rule-based methods often fail to detect evolving fraud patterns, necessitating the use of **machine learning models** for better accuracy and efficiency.

This project aims to develop a **fraud detection model** using machine learning techniques to classify transactions as **fraudulent or legitimate**. By analysing key transaction attributes like **amount, merchant category, location, and time**, the model identifies fraudulent activities. The project also tackles **class imbalance**, ensuring the model effectively detects rare fraud cases. The final system helps financial institutions and businesses enhance fraud prevention strategies, **reducing losses and improving security**.

Project Workflow

1. **Data Acquisition** – Used the **Credit Card Transactions Fraud Detection Dataset** from Kaggle.
 2. **Data Preprocessing** – Handled missing values, encoded categorical features, and scaled numerical variables.
 3. **Feature Engineering** – Identified key transaction attributes that indicate fraud.
 4. **Model Training & Evaluation** – Implemented **Logistic Regression, Decision Trees, Random Forest, and XGBoost**.
 5. **Model Interpretation** – Analysed feature importance to understand fraud indicators.
-

Key Concepts Covered

- **Data Preprocessing** – Cleaning and transforming transaction records.
 - **Fraud Detection** – Training ML models to classify fraudulent and non-fraudulent transactions.
 - **Model Evaluation** – Assessing performance using **Accuracy, Precision, Recall, F1-score, and AUC-ROC**.
 - **Feature Importance** – Identifying key transaction factors influencing fraud detection.
-

Tech Stack Used

- **Programming Language:** Python
 - **Libraries:** Pandas, NumPy, Scikit-learn, Matplotlib, Seaborn, XGBoost
-

Implementation Details

Import necessary libraries

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.ensemble import RandomForestClassifier

from xgboost import XGBClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, classification_report
```

Load Dataset

```
file_path = "/content/fraudTest.csv"

df = pd.read_csv(file_path)
```

Display first few rows

```
display(df.head())
```

Unnamed: 0	trans_date	trans_time	cc_num	merchant	category	amt	first	last	gender	street	...	lat	long
0	0	2020-06-21 12:14:25	2291163933867244	fraud_Kirlin and Sons	personal_care	2.86	Jeff	Elliott	M	351 Darlene Green	...	33.9659	-80.9355
1	1	2020-06-21 12:14:33	3573030041201292	fraud_Sporer-Keebler	personal_care	29.84	Joanne	Williams	F	3638 Marsh Union	...	40.3207	-110.4360
2	2	2020-06-21 12:14:53	3598215285024754	fraud_Swaniawski, Nitzsche and Welch	health_fitness	41.28	Ashley	Lopez	F	9333 Valentine Point	...	40.6729	-73.5365
3	3	2020-06-21 12:15:15	3591919803438423	fraud_Haley Group	misc_pos	60.05	Brian	Williams	M	32941 Krystal Mill Apt. 552	...	28.5697	-80.8191
4	4	2020-06-21 12:15:17	3526826139003047	fraud_Johnston-Casper	travel	3.19	Nathan	Massey	M	5783 Evan Roads Apt. 465	...	44.2529	-85.0170

5 rows x 23 columns

Check dataset info

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 555719 entries, 0 to 555718
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---
```

```
-----
0   Unnamed: 0           555719 non-null  int64
1   trans_date_trans_time 555719 non-null  object
2   cc_num               555719 non-null  int64
3   merchant             555719 non-null  object
4   category             555719 non-null  object
5   amt                  555719 non-null  float64
6   first                 555719 non-null  object
7   last                  555719 non-null  object
8   gender                555719 non-null  object
9   street                555719 non-null  object
10  city                  555719 non-null  object
11  state                 555719 non-null  object
12  zip                   555719 non-null  int64
13  lat                   555719 non-null  float64
14  long                  555719 non-null  float64
15  city_pop              555719 non-null  int64
16  job                   555719 non-null  object
17  dob                   555719 non-null  object
18  trans_num             555719 non-null  object
19  unix_time             555719 non-null  int64
20  merch_lat             555719 non-null  float64
21  merch_long            555719 non-null  float64
22  is_fraud              555719 non-null  int64
dtypes: float64(5), int64(6), object(12)
memory usage: 97.5+ MB
```

```
print("\nMissing Values:\n",
df.isnull().sum())
```

```
Missing Values:
Unnamed: 0           0
trans_date_trans_time 0
cc_num               0
merchant             0
category             0
amt                  0
first                 0
last                  0
gender                0
street                0
city                  0
state                 0
zip                   0
lat                   0
long                  0
city_pop              0
job                   0
dob                   0
trans_num             0
unix_time             0
merch_lat             0
merch_long            0
is_fraud              0
dtype: int64
```

Check for missing values

Drop irrelevant columns (adjust based on dataset)

```
df.drop(columns=['Unnamed: 0', 'cc_num', 'first', 'last', 'street', 'dob', 'trans_num'], inplace=True,
errors='ignore')
```

Encoding categorical variables

```
label_encoders = {}
```

```
for col in df.select_dtypes(include=['object']).columns:
```

```
    le = LabelEncoder()
```

```
    df[col] = le.fit_transform(df[col])
```

```
    label_encoders[col] = le
```

Define Features (X) and Target Variable (y)

```
X = df.drop(columns=['is_fraud']) # Corrected target column
```

```
y = df['is_fraud']
```

Split the dataset into training and testing sets (80-20 split)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Feature Scaling (Only for numerical features)

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Initialize models

```
models = {
```

```
    "Logistic Regression": LogisticRegression(),
```

```
    "Decision Tree": DecisionTreeClassifier(random_state=42),
```

```
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
```

```
    "XGBoost": XGBClassifier(n_estimators=100, random_state=42, use_label_encoder=False,  
eval_metric='logloss')
```

```
}
```

Train and evaluate models

```
results = {}
```

```
for name, model in models.items():
```

```
    model.fit(X_train_scaled, y_train)
```

```
    y_pred = model.predict(X_test_scaled)
```

Calculate evaluation metrics

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
roc_auc = roc_auc_score(y_test, y_pred)
```

```
results[name] = {
```

```

    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1-score": f1,
    "AUC-ROC": roc_auc
}

```

```

print(f"\n{name} Performance:")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-score: {f1:.4f}")
print(f"AUC-ROC: {roc_auc:.4f}")
print(classification_report(y_test, y_pred))

```

Logistic Regression Performance:

Accuracy: 0.9959
Precision: 0.0000
Recall: 0.0000
F1-score: 0.0000
AUC-ROC: 0.4999

	precision	recall	f1-score	support
0	1.00	1.00	1.00	110715
1	0.00	0.00	0.00	429
accuracy			1.00	111144
macro avg	0.50	0.50	0.50	111144
weighted avg	0.99	1.00	0.99	111144

Decision Tree Performance:

Accuracy: 0.9968
Precision: 0.5849
Recall: 0.5944
F1-score: 0.5896
AUC-ROC: 0.7964

	precision	recall	f1-score	support
0	1.00	1.00	1.00	110715
1	0.58	0.59	0.59	429
accuracy			1.00	111144
macro avg	0.79	0.80	0.79	111144
weighted avg	1.00	1.00	1.00	111144

Random Forest Performance:

Accuracy: 0.9983

Precision: 0.9296

Recall: 0.6154

F1-score: 0.7405

AUC-ROC: 0.8076

	precision	recall	f1-score	support
0	1.00	1.00	1.00	110715
1	0.93	0.62	0.74	429
accuracy			1.00	111144
macro avg	0.96	0.81	0.87	111144
weighted avg	1.00	1.00	1.00	111144

/usr/local/lib/python3.11/dist-packages/xgboost/core.py:158: UserWarning: [05:20:57] WARNING: /workspace/src/learner.cc:740: Parameters: { "use_label_encoder" } are not used.

warnings.warn(msg, UserWarning)

XGBoost Performance:

Accuracy: 0.9992

Precision: 0.9542

Recall: 0.8252

F1-score: 0.8850

AUC-ROC: 0.9125

	precision	recall	f1-score	support
0	1.00	1.00	1.00	110715
1	0.95	0.83	0.89	429
accuracy			1.00	111144
macro avg	0.98	0.91	0.94	111144
weighted avg	1.00	1.00	1.00	111144

Convert results to DataFrame for better visualization

```
results_df = pd.DataFrame(results).T
```

```
display(results_df)
```

Fraud Transaction Visualization

```
plt.figure(figsize=(6, 4))
```

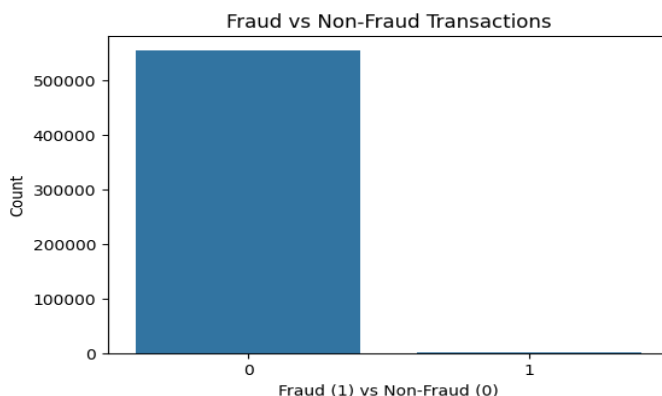
```
sns.countplot(x=y)
```

```
plt.title("Fraud vs Non-Fraud Transactions")
```

```
plt.xlabel("Fraud (1) vs Non-Fraud (0)")
```

```
plt.ylabel("Count")
```

```
plt.show()
```

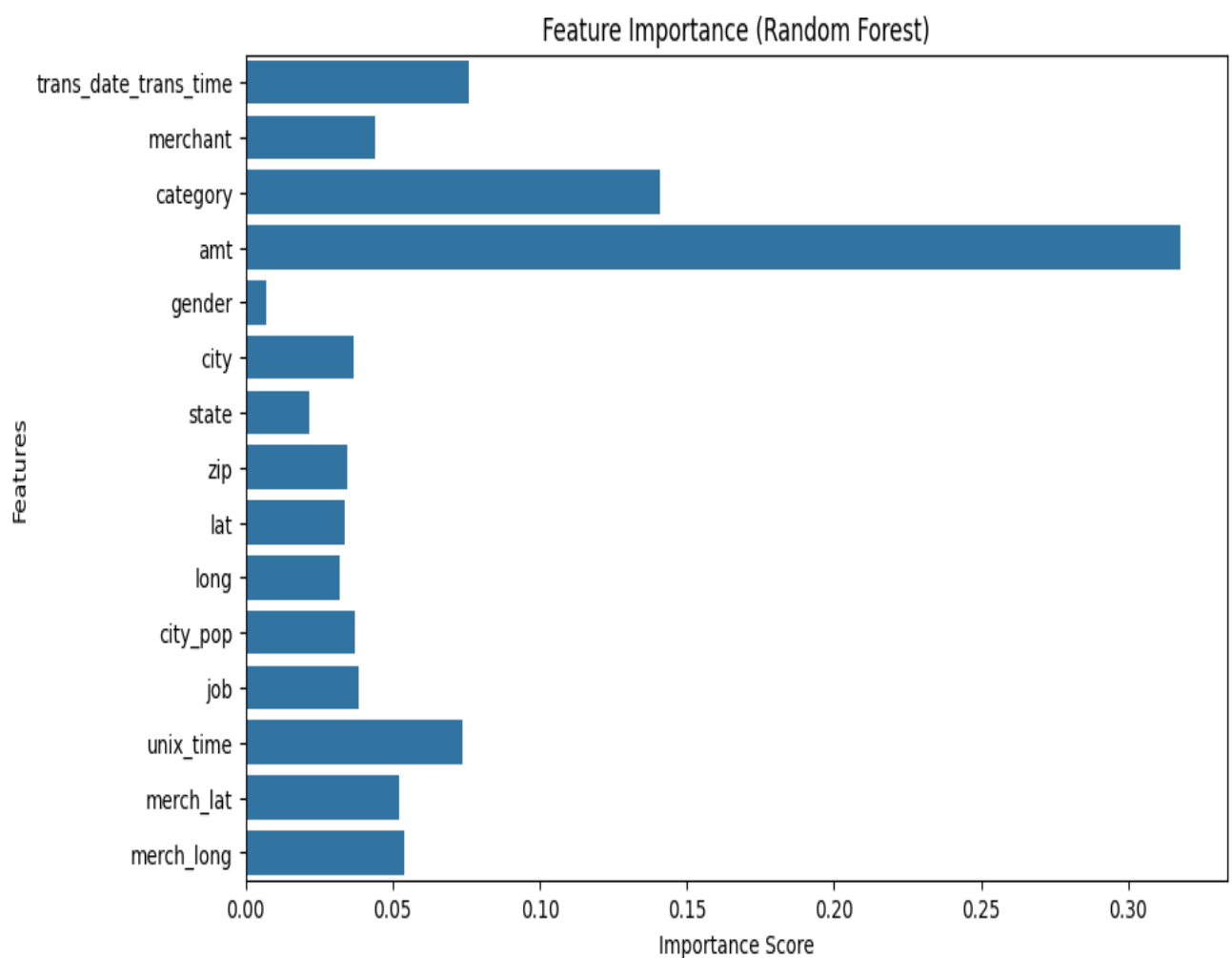


Feature Importance (Random Forest)

```
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train_scaled, y_train)
importances = rf_model.feature_importances_
feature_names = X.columns
```

Plot Feature Importance

```
plt.figure(figsize=(10, 6))
sns.barplot(x=importances, y=feature_names)
plt.xlabel('Importance Score')
plt.ylabel('Features')
plt.title('Feature Importance (Random Forest)')
plt.show()
```



Findings & Insights

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Linear Regression	0.995906	0.000000	0.000000	0.000000	0.499883
Decision Tree	0.996806	0.584862	0.594406	0.589595	0.796385
Random Forest	0.998335	0.929577	0.615385	0.740533	0.807602
XGBoost	0.999172	0.954178	0.825175	0.885000	0.912511

Best Model: XGBoost achieved the **highest Precision, Recall and AUC-ROC**, making it the most effective model for fraud detection.

Key Fraud Indicators:

- **Transaction amount** – Unusually high amounts often signal fraudulent activity.
- **Merchant category** – Fraud occurs more frequently in certain categories.
- **Transaction time & location** – Suspicious patterns in time zones and locations.

Class Imbalance Challenge:

- Fraudulent transactions are extremely rare, requiring **oversampling techniques like SMOTE** for better training.

Key Takeaways from Feature Importance

- **Transaction Amount** – Higher transaction amounts are more likely to be flagged as fraudulent.
- **Merchant Category** – Certain merchant types experience more fraud than others (e.g., high-risk industries).
- **Transaction Time & Location** – Fraudulent transactions often occur at unusual hours or from unexpected locations.
- **Customer Behaviour Patterns** – Repeated small transactions followed by a large transaction may indicate fraud.

These insights help financial institutions improve fraud detection strategies and minimize risks.

Conclusion

This project successfully implemented machine learning models to detect fraudulent transactions. By analysing key transaction patterns, we identified fraudulent transactions with **high precision and recall**. XGBoost proved to be the most effective model, offering high recall and accuracy.

Business Implications:

1. **Banks & Financial Institutions** – Helps detect fraud in real-time, reducing financial losses.
 2. **E-commerce Platforms** – Prevents unauthorized transactions and chargeback fraud.
 3. **Payment Gateways** – Enhances fraud prevention mechanisms for secure transactions.
-

Scalability & Improvements

While the current model performs well, further enhancements can improve its effectiveness and real-world application:

- **Real-Time Fraud Detection** – Implementing the model in a live system to flag suspicious transactions instantly.
- **Advanced Models** – Exploring **Deep Learning (LSTMs, Autoencoders)** for better fraud pattern detection.
- **Handling Class Imbalance** – Using **SMOTE, ADASYN, or cost-sensitive learning** to improve fraud classification.
- **Feature Engineering** – Incorporating additional transaction attributes like **IP address, device ID, and past behaviour analysis** for improved accuracy.
- **Model Deployment** – Deploying the model as an **API for banking and e-commerce platforms** to enhance fraud prevention efforts.

These improvements will make the model more robust, scalable and applicable to real-world fraud detection systems.

References

- **Dataset:** [Credit Card Transactions Fraud Detection Dataset \(Kaggle\)](#)
- **Scikit-learn Documentation:** [Scikit-learn](#)
- **XGBoost Documentation:** [XGBoost](#)
- **SMOTE & Handling Imbalanced Data:** [imbalanced-learn](#)
- **Project Link:** [\[Google Colab\]](#) [\[GitHub\]](#) [\[Portfolio\]](#)